Projeto e Análise de Algoritmos — Lista de Exercícios II

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

---

> **Convenção de custo.** O custo é medido em *operações primitivas* no modelo RAM, atribuindo 1 unidade a cada atribuição, comparação e incremento, conforme o guia de análise fornecido no enunciado. **O custo de um laço inclui os testes de condição**, que ocorrem mesmo quando o corpo não executa — contar apenas o corpo mais interno leva a resposta errada. Escreve-se n^* para pontos de cruzamento entre duas funções de custo, símbolo distinto do n_0 que aparece nas definições assintóticas: são constantes independentes.

> Cabeçalho e instruções da lista (transcrição literal)
>
> Instituto Federal de Educação, Ciência e Tecnologia · Curso: Bacharelado em Ciência da Computação · Disciplina: Projeto e Análise de Algoritmos · Professor: Mário Luiz Rodrigues Oliveira · Atividade: Lista Exercícios II · Formiga,MG, 16 de agosto de 2026
>
> **Instruções:**
>
> 1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.
> 2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.
> 3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1

> Enunciado
>
> Considere as informações abaixo e responda o que se pede. Justifique suas respostas à luz da teoria da complexidade de algoritmos. Questão inspirada em exercícios análogos propostos em [BRASSARD and BRATLEY, 1996].
>
> a. Considere os algoritmos A e B com funções de complexidade no pior caso, respectivamente, $\theta(n^3)$ e $\theta(n^2)$. Em alguns conjuntos de dados, o tempo de execução do algoritmo A é *menor* que o tempo de execução do algoritmo B, quando testados no mesmo ambiente (mesma arquitetura, mesmo sistema operacional, mesma linguagem de programação e mesmo compilador). Forneça três possíveis explicações para esse fenômeno.
> b. Um pesquisador tem a sua disposição 2 algoritmos para resolver um determinado problema. Um algoritmo possui função de complexidade $O(n^2)$ e outro $O(n^3)$. Esse pesquisador escolheu o algoritmo com complexidade $O(n^3)$. Cite 4 razões que poderiam ter motivado essa opção.
> c. Considere os algoritmos A e B para resolver um determinado problema. O algoritmo A exige $10^8 * n$ instruções e o algoritmo B exige $10 * n^2$ instruções. Pode-se afirmar que o algoritmo A é sempre preferível em relação ao algoritmo B?

(a) Três explicações

As três abaixo são **mecanismos distintos**: a primeira é sobre os fatores constantes, a segunda sobre *quais entradas* foram testadas, e a terceira sobre o modelo de custo não corresponder à máquina real.

(i) Constantes ocultas e termos de menor ordem. Θ só afirma a existência de constantes, sem fixar seus valores: $T_A(n) = \Theta(n^3)$ significa apenas que existem $a_1, a_2, n_0 > 0$ com $a_1 n^3 \le T_A(n) \le a_2 n^3$ para $n \ge n_0$.

Tome $T_A(n) = n^3 + 10$ e $T_B(n) = 1000n^2$. As duas respeitam as classes dadas, e

$$T_A(n) < T_B(n) \iff n^3 + 10 < 1000n^2, \quad \text{verdadeiro para todo } 1 \le n \le 999.$$

Em $n = 999$: A faz 997 003 009 operações e B faz 998 001 000 — A vence. Em $n = 1000$ a ordem se inverte (1 000 000 010 contra 1 000 000 000). O cruzamento está em $n^* \approx 1000$, e ele é determinado pelas *constantes*, não pelo n_0 das definições — que pode perfeitamente valer 1.

(ii) Θ descreve o pior caso, e os dados testados podem não ser o pior caso de A. Aqui o mecanismo é outro: não são as constantes, é *a escolha das entradas*.

$T_A(n) = \Theta(n^3)$ no pior caso limita o *máximo* sobre as entradas de tamanho n; não diz nada sobre uma entrada específica. Se A for adaptativo — o *quicksort*, por exemplo, só degrada para $\Theta(n^2)$ com pivôs ruins — pode existir uma família de entradas em que o custo efetivo *de A* seja $c \cdot n + O(1)$. Se, ao mesmo tempo, o conjunto testado for justamente o pior caso de B, cujo custo ali é $d \cdot n^2$, então A é mais rápido sempre que

$$c \cdot n + O(1) < d \cdot n^2, \text{ o que vale para todo } n \text{ suficientemente grande.}$$

> Repare que o limiar *não* é simplesmente $n > c/d$: com um termo aditivo $e > 0$, a condição $c \cdot n + e < d \cdot n^2$ só vale a partir de $n > (c + \sqrt{(c^2 + 4de)})/(2d)$. Com $c = 3$, $d = 1$, $e = 100$, isso dá $n \ge 12$, e não $n > 3$. O que importa aqui é só que $c \cdot n + O(1) = o(n^2)$, logo existe um ponto a partir do qual A vence.

Não há contradição: as duas afirmações falam de conjuntos de entradas diferentes. Uma cota de pior caso é um teto sobre o máximo, e nada obriga A a gastar n^3 nesta família em particular.

(iii) O modelo RAM não captura a hierarquia de memória. O terceiro mecanismo é que o modelo de custo está incompleto para a máquina real.

O modelo RAM cobra $O(1)$ por acesso à memória, independentemente de *onde* o dado esteja. Na máquina real, um acesso em cache custa da ordem de 1 ns e um acesso à memória principal custa da ordem de 100 ns — duas ordens de grandeza. Um algoritmo $\Theta(n^3)$ com acesso sequencial (favorável à pré-busca e à localidade de cache) pode executar em menos tempo de parede que um $\Theta(n^2)$ com acesso aleatório, mesmo realizando mais operações — **até o ponto de cruzamento induzido por esse fator**.

> O ganho é um fator *constante*, e fator constante não inverte $\Theta(n^3)$ contra $\Theta(n^2)$ para todo n. Comparando $n^3/100$ com n^2, os dois empatam exatamente em $n = 100$ (10 000 de cada lado) e a partir daí o cúbico volta a perder — em $n = 500$ já perde por cinco vezes.

As duas classificações continuam corretas: o que falha é a tradução de *número de operações* para *tempo*, que o modelo assume proporcional.

(b) Quatro razões

As quatro apoiam-se em eixos independentes: o tamanho das instâncias, a justeza da cota, os fatores constantes, e recursos que não são tempo.

(i) O tamanho das instâncias é limitado pela natureza do problema. Se a aplicação garante $n \le 100$ sempre — por exemplo, um tabuleiro de tamanho fixo, um número de sensores instalados, um grau máximo de polinômio —, o regime assintótico *nunca é atingido*. A garantia "para todo $n \ge n_0$" pode simplesmente não se aplicar a nenhuma entrada real. Note que, com $n \le 100$, o custo absoluto dos dois é pequeno ($n^3 \le 10^6$ operações, milissegundos), de modo que a classe assintótica deixa de ser o critério de decisão e os outros passam a decidir. Não é que o cúbico ganhe: com constantes iguais ele perde em todo $n \ge 2$ — e em $n = 100$ perde por um fator 100. O que muda é que perder por 100× algo que leva microssegundos deixa de importar.

(ii) O é apenas teto — a cota $O(n^3)$ pode ser frouxa. Este eixo é sobre a qualidade da *análise*, não do algoritmo. Todo $f \in O(n^2)$ também está em $O(n^3)$, pois $n^2 \le n^3$ para $n \ge 1$. Logo $O(n^3)$ pode ser apenas a melhor cota que alguém conseguiu *provar*, enquanto o algoritmo é de fato $\Theta(n^2)$ ou melhor. Comparar dois algoritmos pelos seus O não é comparação válida: O limita por cima, e um teto frouxo não é evidência de lentidão.

(iii) Os fatores constantes são muito desiguais. Aqui n não é limitado e as cotas são justas; o que decide são as constantes. Com $T_1(n) = 5000n^2$ e $T_2(n) = 2n^3$:

$$2n^3 < 5000n^2 \iff n < 2500.$$

Ou seja, o algoritmo cúbico faz *menos* operações para todo $1 \le n \le 2499$. Em $n = 2500$ há **empate exato** (31 250 000 000 operações de cada lado), e só a partir de $n = 2501$ o quadrático passa à frente. Se a faixa de interesse do pesquisador estiver abaixo de 2500, a escolha do cúbico é a correta.

(iv) Critérios que não são tempo de execução. O modelo RAM contabiliza apenas operações; a decisão de engenharia envolve outros recursos. O algoritmo $O(n^3)$ pode usar $O(1)$ de espaço enquanto o $O(n^2)$ usa $\Theta(n^2)$ e simplesmente não cabe em memória; pode ser muito mais simples de implementar e manter, portanto menos sujeito a erro; pode ser numericamente mais estável; pode já estar implementado, validado e em produção; ou pode ser trivialmente paralelizável, reduzindo o tempo de parede apesar de mais operações totais.

(c) Não

Aqui $10^8 n$ e $10n^2$ são contagens *exatas* de instruções, e não cotas assintóticas — então a comparação é direta, sem constantes ocultas.

$$10^8 \cdot n < 10 \cdot n^2 \iff 10^8 < 10n \iff n > 10^7 \quad \text{(dividindo por } n \ge 1\text{, que é positivo).}$$

n	$A = 10^8 n$	$B = 10n^2$	preferível
10^6	10^{14}	10^{13}	**B** — dez vezes menos instruções
10^7	10^{15}	10^{15}	empate exato
10^8	10^{16}	10^{17}	**A**

Assintoticamente $T_A = \Theta(n)$ e $T_B = \Theta(n^2)$, logo A vence para n suficientemente grande. Mas "suficientemente grande" aqui significa $n > 10^7$, um limiar altíssimo causado pela constante 10^8. Como a preferência depende do tamanho da instância, **não** se pode afirmar que A é sempre preferível.

Questão 2

> Enunciado
>
> Considere a teoria da complexidade de algoritmos e responda o que se pede. Justifique formalmente suas respostas. Respostas sem justificativas não serão consideradas.
>
> a. Explique por que a frase "O tempo de execução do algoritmo A é no mínimo $O(n^2)$"não faz sentido.
> b. Se um algoritmo A tem tempo de execução $\theta(n^2)$ sobre qualquer entrada de tamanho n, ele pode ter tempo de execução $\Omega(n\log n)$ no pior caso?

(a) A frase mistura piso com teto — e, na leitura mais generosa, é vazia

1. Incoerência de sentido. Por definição, $T(n) = O(n^2)$ significa que existem $c > 0$ e $n_0 > 0$ com $0 \le T(n) \le c \cdot n^2$ para todo $n \ge n_0$. É um **teto**: diz que T não passa de $c \cdot n^2$. Já "no mínimo" pede uma afirmação da forma $T(n) \ge (algo)$, que é exatamente o papel de Ω. Não existe, na definição de O, nenhuma desigualdade do tipo $T(n) \ge \ldots$ para "no mínimo" qualificar.

2. A frase é vazia. Pode-se tentar salvá-la lendo "no mínimo $O(n^2)$" como "$T(n) = O(g(n))$ para alguma g pelo menos tão grande quanto n^2". Mesmo assim não informa nada, porque as classes O são ordenadas por inclusão e crescem *para cima*:

$$O(n^2) \subset O(n^3) \subset O(2^n) \subset \cdots$$

Logo "pelo menos $O(n^2)$" aponta para as classes *maiores*, que são afirmações mais **fracas**. Formalmente: seja T o tempo de um algoritmo qualquer e defina $g(n) = T(n) + n^2$. Então $g(n) \ge n^2$ para todo $n \ge 1$, e vale $T(n) = O(g(n))$ com $c = 1$ e $n_0 = 1$, pois $0 \le T(n) \le T(n) + n^2 = 1 \cdot g(n)$. Como T era arbitrária, *todo* algoritmo satisfaz a frase. Uma afirmação satisfeita por todos os objetos do universo não separa nada.

Redação correta: "o tempo de execução de A é $\Omega(n^2)$".

(b) Sim, pode — e de fato tem. Não há contradição.

Hipótese. Para toda entrada de tamanho n, o custo de A é $\Theta(n^2)$: existem $c_1, c_2 > 0$ e n_0 com $0 \le c_1 n^2 \le T(n) \le c_2 n^2$ para $n \ge n_0$.

Passo 1 — o pior caso herda os limites. Seja $T_{pior}(n)$ o máximo de $T(x)$ sobre as entradas de tamanho n. O máximo de valores todos contidos em $[c_1 n^2, c_2 n^2]$ também pertence a esse intervalo, logo $T_{pior}(n) = \Theta(n^2)$.

Passo 2 — lema: $\lg n \le n$ para $n \ge 1$. Basta $2^n \ge n$, por indução. Base: $2^1 = 2 \ge 1$. Passo: se $2^n \ge n$ com $n \ge 1$, então $2^{n+1} = 2 \cdot 2^n \ge 2n = n + n \ge n + 1$. Como lg é crescente, $n = \lg(2^n) \ge \lg n$.

Passo 3 — testemunhas. Multiplicando o lema por $n > 0$: $n \cdot \lg n \le n^2$. Tome $c = c_1$ e $n_0' = \max(n_0, 2)$ (o 2 evita o caso degenerado $\lg 1 = 0$). Para todo $n \ge n_0'$:

$$T_{pior}(n) \ge c_1 n^2 \ge c_1 \cdot n \lg n, \quad \text{isto é,} \quad T_{pior}(n) = \Omega(n \lg n). \ \blacksquare$$

Por que não é contraditório. $\Omega(n \lg n)$ *não* significa "o algoritmo gasta n lg n". Significa "gasta pelo menos da ordem de n lg n" — e nada impede que gaste muito mais. Como n^2 domina $n \lg n$, um custo $\Theta(n^2)$ satisfaz o piso com folga.

> **A afirmação é verdadeira, mas frouxa.** Ela não é a cota inferior mais firme, que é $\Omega(n^2)$. De fato T_{pior} não é $\Theta(n \lg n)$: se fosse, existiria c_3 com $c_1 n^2 \le c_3 \cdot n \lg n$, ou seja $n / \lg n \le c_3 / c_1$ para todo n grande — falso, pois $n / \lg n$ é ilimitada (em $n = 2^k$ vale $2^k / k$, que cresce sem limite).

Questão 3

> Enunciado
>
> Prove que as sentenças abaixo são verdadeiras ou falsas, usando para isso as definições das notações assintóticas e/ou contra exemplos. Considere que as funções f(n) e g(n) são assintoticamente não negativas.
>
> a. $f(n) \in O(g(n))$ se e somente se $g(n) \in \Omega(f(n))$
> b. Se $f(n) \in O(g(n))$ então $g(n) \in O(f(n))$.
> c. Se $f(n) \in O(u(n))$ e $g(n) \in O(v(n))$ então $f(n) + g(n) \in O(u(n) + v(n))$.
> d. Se $f(n) \in O(g(n))$ e $g(n) \in O(f(n))$ então $f(n) \in \theta(g(n))$.

Em provas de propriedade *geral* as constantes não podem ser apenas declaradas: têm de ser **construídas** a partir das que a hipótese fornece, e nomeadas sem colisão de símbolos.

(a) Verdadeira

As duas notações são a *mesma* desigualdade lida dos dois lados. Sendo um "se e somente se", provam-se as duas direções.

($\Longrightarrow$) Se $f \in O(g)$, existem $c > 0$ e n_0 com $0 \le f(n) \le c \cdot g(n)$ para $n \ge n_0$. Fixe $d = 1/c$ e $m_0 = n_0$ — legítimo porque $c > 0$ estritamente, e nenhuma delas depende de n. Dividindo por $c > 0$: $d \cdot f(n) \le g(n)$. Com $f(n) \ge 0$ e $d > 0$ vem $d \cdot f(n) \ge 0$. Logo $0 \le d \cdot f(n) \le g(n)$, que é $g \in \Omega(f)$.

($\Longleftarrow$) Se $g \in \Omega(f)$, existem $d' > 0$ e m_0' com $0 \le d' \cdot f(n) \le g(n)$. Fixe $c' = 1/d'$ e $n_0' = m_0'$. Dividindo por $d' > 0$: $f(n) \le c' \cdot g(n)$; e de $0 \le d' \cdot f(n)$ segue $f(n) \ge 0$. Logo $f \in O(g)$. $\blacksquare$

(b) Falsa

Contraexemplo: $f(n) = n$ e $g(n) = n^2$, ambas não negativas para $n \ge 1$.

A hipótese vale. Com $c = 1$ e $n_0 = 1$: de $n \ge 1$ vem $n^2 \ge n$, logo $0 \le n \le 1 \cdot n^2$ para $n \ge 1$, isto é, $f \in O(g)$.

A conclusão falha. Suponha por absurdo que existam $c > 0$ e n_0 com $0 \le n^2 \le c \cdot n$ para $n \ge n_0$. Dividindo por $n > 0$: $n \le c$ para todo $n \ge \max(n_0, 1)$. Tome a testemunha $n^* = \max(n_0, 1) + \lfloor c \rfloor + 1$: ela cumpre $n^* \ge \max(n_0, 1)$, então a desigualdade deveria valer, mas $n^* > c$. Contradição. Logo $g \notin O(f)$. $\blacksquare$

> Coerente com (a): $f \in O(g)$ garante apenas $g \in \Omega(f)$. Nada obriga f a ser também teto de g.

(c) Verdadeira

De $f \in O(u)$: existem $c_1 > 0$, n_1 com $0 \le f(n) \le c_1 u(n)$ para $n \ge n_1$. De $g \in O(v)$: existem $c_2 > 0$, n_2 com $0 \le g(n) \le c_2 v(n)$ para $n \ge n_2$.

Construa $C = máx(c_1, c_2)$ e $N = máx(n_1, n_2)$. Para todo $n \ge N$:

$$f(n) + g(n) \le c_1 u(n) + c_2 v(n) \le C \cdot u(n) + C \cdot v(n) = C \cdot (u(n) + v(n)).$$

O passo $c_1 u(n) \le C \cdot u(n)$ exige $u(n) \ge 0$, e isso sai da própria hipótese — o enunciado só fala de f e g: de $0 \le f(n) \le c_1 u(n)$ com $c_1 > 0$ vem $u(n) \ge 0$ para todo $n \ge n_1$; do mesmo modo $v(n) \ge 0$ para $n \ge n_2$, e as duas valem para $n \ge N$. E $f(n) + g(n) \ge 0$ porque ambas são não negativas. Logo $f + g \in O(u + v)$ com testemunhas C e N. ∎

(d) Verdadeira — é o Teorema 3.1 do CLRS

De $f \in O(g)$: existem $b > 0$, n_1 com $0 \le f(n) \le b \cdot g(n)$ para $n \ge n_1$.

De $g \in O(f)$: existem $a > 0$, n_2 com $0 \le g(n) \le a \cdot f(n)$ para $n \ge n_2$; dividindo por $a > 0$, $f(n) \ge (1/a) \cdot g(n)$.

Construa $c_1 = 1/a$, $c_2 = b$ e $n_0 = máx(n_1, n_2)$. Para todo $n \ge n_0$:

$$0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n),$$

que é exatamente a definição de $f \in \Theta(g)$. ∎

> Note que os símbolos foram batizados a e b justamente para não colidirem com c_1 e c_2, que são as constantes *construídas* para o Θ.

Questão 4

> Enunciado
>
> Considere n representando um valor variável e k é um valor constante. Analise os códigos abaixo e encontre a complexidade assintótica em cada caso usando a notação *Big O*. Dê um limite assintótico tão firme quanto possível. Ao realizar a análise, considere o pior caso. Justifique suas respostas. Respostas sem justificativas não serão consideradas.
>
> Os cinco códigos estão transcritos literalmente, cada um no início do seu item abaixo.

Código 1 — O(log n), e o limite é justo: Θ(log n)

> Enunciado — código 1 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  i = 1;
3  While ( i <= n )
4  {
5      soma = soma + 1;
6      i = i * 10;
7  }
```

Após t multiplicações vale $i = 10^t$ (começando em $i = 1$). O corpo executa enquanto

$$10^t \le n \iff t \le \log_{10} n,$$

e como t é inteiro começando em 0, o corpo executa exatamente $\lfloor \log_{10} n \rfloor + 1$ vezes. Somando o custo constante de fora do laço e o teste final que falha (1 unidade), $T(n) = a + b \cdot (\lfloor \log_{10} n \rfloor + 1) + 1$ — na contagem do guia, $a = 2$ e $b = 3$, isto é, $T(n) = 3(\lfloor \log_{10} n \rfloor + 1) + 3$.

Teto. Para $n \ge 10$ vale $\log_{10} n \ge 1$, logo $\lfloor \log_{10} n \rfloor + 1 \le 2 \log_{10} n \le 2 \lg n$. Com $c = a + 4b$ e $n_0 = 10$ obtém-se $T(n) \le c \cdot \lg n$. *Piso.* $\lfloor \log_{10} n \rfloor + 1 > \lg n / \lg 10$, logo $c_1 = b / \lg 10$ e $n_0 = 1$. Portanto $\Theta(\log n)$.

Código 2 — O(log n), e o limite é justo: Θ(log n)

> Enunciado — código 2 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  i = n;
3  While ( i > 0 )
4  {
5      soma = soma + i;
6      i = i / 2; // considere divisao inteira
7  }
```

Com divisão inteira, após t divisões vale $i = \lfloor n / 2^t \rfloor$. O corpo executa enquanto $\lfloor n / 2^t \rfloor \ge 1$, isto é, $2^t \le n$, ou $t \le \lg n$. Logo o corpo executa exatamente $\lfloor \lg n \rfloor + 1$ vezes. Conferência: $n = 1$ dá 1 iteração; $n = 8$ dá 4 ($i = 8, 4, 2, 1$).

Teto. $\lfloor \lg n \rfloor + 1 \le 2 \lg n$ para $n \ge 2$; tome $c = a + 3b$, $n_0 = 2$. *Piso.* $\lfloor \lg n \rfloor + 1 > \lg n$; tome $c_1 = b$, $n_0 = 1$. Portanto $\Theta(\log n)$.

> Os códigos 1 e 2 são logarítmicos por mecanismos *opostos*: no primeiro a variável multiplica por 10 subindo até n; no segundo divide por 2 descendo até 0. As bases são irrelevantes assintoticamente, pois $\log_{10} n = \lg n / \lg 10$.

Código 3 — O(n), e o limite é justo: Θ(n) no pior caso

Enunciado — código 3 (transcrição literal, com a numeração de linhas da lista)

```
1  x = escolha (1,n); // retorna um valor inteiro no intervalo [1..n] em O (1)
2  soma = 0;
3  for (i = 1; i <= n; i = i + 1)
4  {
5      if ( i != x )
6      {
7          soma = soma + i;
8      }
9      else
10     {
11         for (j = 1; j <= i; j = j + 1)
12         {
13             soma = soma + j * x;
14         }
15     }
16 }
```

A condição é `i != x`, então o laço interno está no `else` e só é alcançado quando $i == x$ — o que acontece *exatamente uma vez* em toda a execução, já que x é um único valor de [1..n].

O laço externo custa $a{\cdot}n + a'$. O laço interno roda uma única vez, na iteração $i = x$, com $j = 1..x$, custando $b{\cdot}x + b'$. Logo

$$T(n, x) = a{\cdot}n + b{\cdot}x + d.$$

T é crescente em x, e $x \in [1..n]$, então o **pior caso é x = n**: $T_{pior}(n) = (a + b)n + d$. Testemunhas: $c_1 = a$, $c_2 = a + b + d$, $n_0 = 1$. Portanto $\Theta(n)$.

> **Por que não é Θ(n²).** O somatório $\Sigma_{i=1..n}\, i = n(n+1)/2$, que produziria custo quadrático, só apareceria se o laço interno rodasse para *todo* i. Aqui ele roda para um único i, e o somatório colapsa num único termo $x \le n$.

Código 4 — O(n²), e o limite é justo: Θ(n²) no pior caso

Enunciado — código 4 (transcrição literal, com a numeração de linhas da lista)

```
1  x = escolha (1,n); // retorna um valor inteiro no intervalo [1..n] em O(1)
2  soma = 0;;
3  for (i = 1; i <= n; i = i + 1)
4  {
5      if (i == x)
6      {
7          soma = soma + i;
8      }
9      else
10     {
11         for (j = 1; j <= i; j = j + 1)
12         {
13             soma = soma + j * x;
14         }
15     }
16 }
```

A única diferença em relação ao código 3 é a condição, agora `i == x` — e ela inverte a análise. O laço interno continua no `else`, mas agora o `else` é alcançado para *todo* $i \neq x$, ou seja, em $n - 1$ das n iterações.

Número total de execuções do corpo interno, em forma fechada:

$$\Sigma_{i=1,\, i\neq x}^{n} i = n(n+1)/2 - x.$$

Logo $T(n, x) = A{\cdot}n(n+1)/2 - B{\cdot}x + C{\cdot}n + D$, que é *decrescente* em x: cada unidade a mais em x retira exatamente uma passagem pelo laço interno. O **pior caso é x = 1**. Com $n(n+1)/2 \le n^2$ para $n \ge 1$ obtém-se o teto, e $n(n+1)/2 \ge n^2/2$ o piso. Portanto $\Theta(n^2)$.

Testemunhas. Na contagem do guia (1 unidade por atribuição, teste e incremento), o pior caso custa exatamente $W(n) = T(n, 1) = (3/2)n^2 + (13/2)n$ — conferido por força bruta para $n = 1..149$, e $x = 1$ é o pior em todo n testado. Para $n \ge 1$, $(3/2)n^2 \le W(n) \le (3/2)n^2 + (13/2)n^2 = 8n^2$: $c_1 = 3/2$, $c_2 = 8$, $n_0 = 1$ (em $n = 1$: $1{,}5 \le 8 \le 8$).

> **Os piores casos são opostos:** $x = n$ no código 3 e $x = 1$ no código 4. Responder os dois igual erra um deles.

Código 5 — O(n log n), e o limite é justo: Θ(n log n)

Enunciado — código 5 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  for (i = 1; i <= n; i = i + 1)
3  {
4      for (j = 1; j <= i; j = 3*j)
5      {
6          for (k = 1; k <= 25; k = k + 1)
7          {
8              soma = soma + 1;
9          }
10     }
11 }
```

O laço mais interno é O(1): ele vai de 1 a **25**, um limite constante que não depende de n — 25 iterações sempre.

O laço do meio é logarítmico: j multiplica por 3 a cada passo, valendo 3^t após t passos. Ele executa enquanto $3^t \le i$, ou seja $\lfloor \log_3 i \rfloor + 1$ vezes.

Somando sobre o laço externo:

$$T(n) = \Theta\left(\sum_{i=1..n} (\lfloor \log_3 i \rfloor + 1) \right) = \Theta(n \log n).$$

Teto. $\lfloor \log_3 i \rfloor + 1 \le \log_3 n + 1 \le 2 \log_3 n$ para $i \le n$ e $n \ge 3$; somando sobre n valores, $T(n) = O(n \log n)$. *Piso.* Para os $n/2$ valores com $i > n/2$ vale $\lfloor \log_3 i \rfloor + 1 > \log_3(n/2)$, logo $T(n) = \Omega(n \log n)$. A base 3 só altera a constante, pois $\log_3 n = \lg n / \lg 3$.

Testemunhas. Na contagem do guia, $T(n) = 4n + 3 + 79 \cdot S(n)$, com $S(n) = \Sigma_{i=1..n} (\lfloor \log_3 i \rfloor + 1)$: cada volta do laço do meio custa 79 (seu teste, $k = 1$, 25 × 3 do laço interno, o teste final dele e $j = 3 \cdot j$) — conferido por força bruta. *Teto:* para $n \ge 3$, $S(n) \le 2n \cdot \log_3 n$ e $4n + 3 \le 7n \le 7n \cdot \log_3 n$, logo $T(n) \le 165 \cdot n \cdot \log_3 n = (165/\lg 3) \cdot n \lg n$. *Piso:* os $\lceil n/2 \rceil \ge n/2$ índices $i > n/2$ têm $\lfloor \log_3 i \rfloor + 1 > \log_3(n/2) \ge \frac{1}{2} \cdot \log_3 n$ (a última desigualdade vale para $n \ge 4$), logo $T(n) \ge 79 \cdot (n/4) \cdot \log_3 n = (79/(4 \lg 3)) \cdot n \lg n$. Testemunhas: $c_1 = 79/(4 \lg 3) \approx 12{,}46$, $c_2 = 165/\lg 3 \approx 104{,}10$, $n_0 = 4$ — em $n = 4$: $99{,}7 \le T(4) = 493 \le 832{,}8$; conferidas por força bruta de $n = 4$ a $2 \cdot 10^5$.

Código	Iterações do laço decisivo	Resposta (Big O, mais firme)	Por quê
1	$\lfloor \log_{10} n \rfloor + 1$	**O(log n)**	i multiplica por 10 subindo até n
2	$\lfloor \lg n \rfloor + 1$	**O(log n)**	i divide por 2 descendo até 0
3	interno roda 1 vez (i = x)	**O(n)**	`if (i != x)` põe o laço no else; pior caso x = n
4	interno roda n − 1 vezes	**O(n²)**	`if (i == x)` inverte tudo; pior caso x = 1
5	$\lfloor \log_3 i \rfloor + 1$ por i	**O(n log n)**	laço de 25 é constante; o do meio é log

Informações que podem ser úteis

Anexo da lista (transcrição literal)

Análise de Algoritmos:

Não há um conjunto de regras para analisar um algoritmo. No entanto, conforme [AHO et al., 1983] podemos seguir o seguinte guia, considerando o modelo computacional *Random Access Machine* (RAM):

1. O custo de execução de uma instrução de atribuição, de leitura ou de escrita é O(1);
2. O custo de execução de uma sequência de instruções é determinado pela instrução de maior custo na sequência de instruções;
3. O custo de execução de um comando de decisão/seleção é composto pelo custo de execução das instruções no corpo do comando de decisão/seleção mais o custo para avaliar a condição. O custo para avaliar a condição é O(1), e
4. O custo de execução de um comando de repetição (laço/loop) é a soma do custo de execução do corpo do comando de repetição adicionado ao custo de avaliar a condição de término do comando de execução, multiplicado pelo número de iterações do comando de repetição. O custo para avaliar a condição de término é O(1).

Somatórios e Logaritmos:

- $\Sigma_{i=1}^{n} (i) = n(n + 1)/2$
- $\Sigma_{i=1}^{n} (a^i) = a(a^n - 1)/(a - 1)$
- $\log (a * b) = \log a + \log b$
- $\log (a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:

$$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$$

onde c é uma constante arbitrária inteira ≥ 2.

Referências

- [AHO et al., 1983] AHO, A. V., HOPCORFT, J. E., and ULLMAN, J. D. (1983). Data Structures and Algorithms. Addison-Wesley.
- [BRASSARD and BRATLEY, 1996] BRASSARD, G. and BRATLEY, P. (1996). Fundamentals of Algorithms. Prentice-Hall.

Sobre esta resolução. As classes assintóticas da Questão 4 foram conferidas por medição: os cinco códigos foram transcritos em Python e todas as operações primitivas contadas por força bruta, com varredura exaustiva de x para confirmar qual valor realiza o pior caso nos códigos 3 e 4. Os pontos de cruzamento citados na Questão 1 foram verificados numericamente, incluindo os empates exatos em $n = 2500$ e $n = 10^7$. Os scripts estão em `resolucoes/verificacao/`. As demonstrações foram submetidas a verificadores independentes em contexto isolado, que receberam apenas o enunciado e a prova final. Lista de estudo, sem entrega — IFMG Campus Formiga, 2026/2.